To use JWT with SQL Server and ef core, we need these packages:

- `Microsoft.AspNetCore.Identity.EntityFrameworkCore`: This package is used for the EF Core implementation of ASP.NET Core Identity.
- `Microsoft.EntityFrameworkCore.SqlServer`: This package is used to connect to SQL Server.
- `Microsoft.EntityFrameworkCore.Tools`: This package is used to enable the necessary EF Core tools.
- `Microsoft.AspNetCore.Authentication.JwtBearer`: This package is used to enable JWT authentication.

****

3. Next, we will add the database context. We will use EF Core to access the database. But first, we need an entity model to represent the user. Create a new folder named `Authentication` and add a new class named `AppUser` to it. The `AppUser` class inherits from the `IdentityUser` class, which is provided by ASP.NET Core Identity, as shown in the following code:

```
public class AppUser : IdentityUser
{
}
```

The `IdentityUser` class already contains the properties that we need to represent a user for most of the scenarios, such as `UserName`, `Email`, `PasswordHash`, `PhoneNumber`, and others.

****

```
using Microsoft.AspNetCore.Identity;

namespace JLokaAuthentication.Authentication
{
    public class AppUser: IdentityUser
    {
    }
}
```

****

For using identity with db context:

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace JLokaAuthentication.Authentication
{
    public class AppDbContext(DbContextOptions<AppDbContext> options, IConfiguration
configuration) : IdentityDbContext<AppUser> (options)
    {
        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        {
            base.OnConfiguring(optionsBuilder);

            optionsBuilder.UseSqlServer(configuration.GetConnectionString("DefaultC
            onnection"));
        }
    }
}
```

****

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    //"DefaultConnection": "Server=DESKTOP-
K9V44R0\\SQLExpress;Database=JLokaDB;Trusted_Connection=true;TrustServerCertifica
te=true;MultipleActiveResultSets=true;"
    "DefaultConnection": "Server=localhost;Database=Itcp;User
ID=sa;Password=P@ssw0rd1;TrustServerCertificate=true;MultipleActiveResultSets=tru
e;"
  }
}
```

****

As we can see, this AppDbContext is purely for ASP.NET Core Identity. If you have other entities in your application, you can create a separate DbContext for them if you want to. You can use the same connection string for both DbContexts.

****

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();

// Add the db context
builder.Services.AddDbContext<AppDbContext>();
// Add the identity model and attach to db context
builder.Services.AddIdentityCore<AppUser>()
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders();
//Add authentication with options
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
}).AddJwtBearer(options =>
{
    var secret = builder.Configuration["JwtConfig:secret"];
    var issuer = builder.Configuration["JwtConfig:ValidIssuer"];
    var audience = builder.Configuration["JwtConfig:ValidAudiences"];

    if (secret is null || issuer is null || audience is null)
    {
        throw new ApplicationException("Jwt is not set in the configuration");
    }

    options.SaveToken = true;
    options.RequireHttpsMetadata = false;
    options.TokenValidationParameters = new TokenValidationParameters()
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidAudience = audience,
        ValidIssuer = issuer,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret)),
    };
});

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapControllers();

app.Run();
```

****

And to make the controller or methods use Authorization:

```
[Authorize]
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot",
"Sweltering", "Scorching"
    };

    private readonly ILogger<WeatherForecastController> _logger;

    public WeatherForecastController(ILogger<WeatherForecastController> logger)
    {
        _logger = logger;
    }

    [HttpGet(Name = "GetWeatherForecast")]
    public IEnumerable<WeatherForecast> Get()
    {
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
        {
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            TemperatureC = Random.Shared.Next(-20, 55),
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
        })
        .ToArray();
    }
}
```

****

To generate a JWT token, we need to provide the following information:

The issuer of the token: This is the server that issues the token.

The audience of the token: This is the server that consumes the token. It can be the same as the issuer or include multiple servers.

The secret key to sign the token: This is used to validate the token.

****

To use the controller for Account, we must inherit from ControllerBase:

```
using JLokaAuthentication.Authentication;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace JLokaAuthentication.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class AccountController(UserManager<AppUser> userManager, IConfiguration
_configuration) : ControllerBase
    {
        [HttpPost("register")]
        public async Task<IActionResult> Register([FromBody] AddOrUpdateAppUserModel
model)
        {
            // check if the data is valid
            if (ModelState.IsValid)
            {
                var existedUser = await userManager.FindByNameAsync(model.UserName);
                if (existedUser != null)
                {
                    ModelState.AddModelError("", "Username already exists");
                    return BadRequest(ModelState);
                }

                // create the user model
                var user = new AppUser()
                {
                    UserName = model.UserName,
                    Email = model.Email,
                    SecurityStamp = Guid.NewGuid().ToString(),
                };

                // try to save the user with password
                var userResult = await userManager.CreateAsync(user,
model.Password);

                // Adding the user to "User" Role
                var roleResult = await userManager.AddToRoleAsync(user,
AppRoles.User);

                // if the result is success return the result
                if (userResult.Succeeded)
                {
                    var token = GenerateToken(model.UserName, user);
                    return Ok(new { token });
                }

                // else return the errors
                foreach (var error in userResult.Errors)
                {
                    ModelState.AddModelError("", error.Description);
```

```
                }
            }
            return BadRequest(ModelState);
        }

        private async Task<string?> GenerateToken(string userName, AppUser user)
        {
            var secret = _configuration["JwtConfig:Secret"];
            var issuer = _configuration["JwtConfig:ValidIssuer"];
            var audience = _configuration["JwtConfig:ValidAudiences"];

            if(secret is null || issuer is null || audience is null)
            {
                throw new ApplicationException("Jwt is not set in the
configuration");
            }
            var signingKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret));
            var tokenHandler = new JwtSecurityTokenHandler();
            var userRoles = await userManager.GetRolesAsync(user);
            var claims = new List<Claim>
            {
                new(ClaimTypes.Name, userName)
            };
            claims.AddRange(userRoles.Select(role => new Claim(ClaimTypes.Role,
role)));

            var tokenDescriptor = new SecurityTokenDescriptor
            {
                Subject = new System.Security.Claims.ClaimsIdentity(claims),
                Expires = DateTime.UtcNow.AddDays(1),
                Issuer = issuer,
                Audience = audience,
                SigningCredentials = new SigningCredentials(signingKey,
SecurityAlgorithms.HmacSha256Signature),
            };
            var securityToken = tokenHandler.CreateToken(tokenDescriptor);
            var token = tokenHandler.WriteToken(securityToken);
            return token;
        }

        [HttpPost("login")]
        public async Task<IActionResult> Login([FromBody] LoginModel model)
        {
            // Get the secret in the configuration
            // Check if the model is valid
            if(ModelState.IsValid)
            {
                var user = await userManager.FindByNameAsync(model.Username);
                if(user != null)
                {
                    if (await userManager.CheckPasswordAsync(user, model.password))
                    {
                        var token = GenerateToken(model.Username, user);
                        return Ok(new { token });
                    }
                }
                // If the user is not found, display an error message
```

```
                ModelState.AddModelError("", "Invalid username or password");
            }
            return BadRequest(ModelState);
        }
    }
}
             ********************************************************************
```

To use the roles with the user:

```
using JLokaAuthentication.Authentication;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

// Add the db context
builder.Services.AddDbContext<AppDbContext>();
// Add the identity model and attach to db context
builder.Services.AddIdentityCore<AppUser>()
    //  If you use the AddIdentity() method, you do not need to call the AddRoles()
method.
    // The AddIdentity() method will call the AddRoles() method internally.
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders();
//Add authentication with options
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
}).AddJwtBearer(options =>
{
    var secret = builder.Configuration["JwtConfig:secret"];
    var issuer = builder.Configuration["JwtConfig:ValidIssuer"];
    var audience = builder.Configuration["JwtConfig:ValidAudiences"];

    if (secret is null || issuer is null || audience is null)
    {
        throw new ApplicationException("Jwt is not set in the configuration");
    }

    options.SaveToken = true;
    options.RequireHttpsMetadata = false;
    options.TokenValidationParameters = new TokenValidationParameters()
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidAudience = audience,
        ValidIssuer = issuer,
```

```
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret)),
    };
});

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new
Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        Description = "JWT Authorization header using the Bearer scheme.Example:
\"Authorization: Bearer {token}\"",
        Name = "Authorization",
        In = Microsoft.OpenApi.Models.ParameterLocation.Header,
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.ApiKey,
        Scheme = "Bearer"
    });
    c.AddSecurityRequirement(new
Microsoft.OpenApi.Models.OpenApiSecurityRequirement()
    {
                {
                    new OpenApiSecurityScheme
                    {
                        Reference = new OpenApiReference
                        {
                            Type = ReferenceType.SecurityScheme,
                            Id = "Bearer"
                        }
                    },
                    new String[] {}
                }
    });
});

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdministratorRole", policy =>
policy.RequireRole(AppRoles.Administrator));
    options.AddPolicy("RequireVipUserRole", policy =>
policy.RequireRole(AppRoles.VipUser));
    options.AddPolicy("RequireUserRole", policy =>
policy.RequireRole(AppRoles.User));
    options.AddPolicy("RequireUserRoleOrVipUserRole", policy =>
policy.RequireRole(AppRoles.User, AppRoles.VipUser));
});

var app = builder.Build();
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
```

```
// Check if the roles exist, if not, create them
using (var serviceScope = app.Services.CreateScope())
{
    var services = serviceScope.ServiceProvider;
    // Ensure the database is created.
    var dbContext = services.GetRequiredService<AppDbContext>();
    //dbContext.Database.EnsureDeleted();
    dbContext.Database.EnsureCreated();

    var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
    if (!await roleManager.RoleExistsAsync(AppRoles.User))
    {
        await roleManager.CreateAsync(new IdentityRole(AppRoles.User));
    }

    if (!await roleManager.RoleExistsAsync(AppRoles.VipUser))
    {
        await roleManager.CreateAsync(new IdentityRole(AppRoles.VipUser));
    }

    if (!await roleManager.RoleExistsAsync(AppRoles.Administrator))
    {
        await roleManager.CreateAsync(new IdentityRole(AppRoles.Administrator));
    }
}

app.MapControllers();
app.Run();
```

****

The user login is:

```
using System.ComponentModel.DataAnnotations;

namespace JLokaAuthentication.Authentication
{
    public class LoginModel
    {
        [Required(ErrorMessage = "Username is required")]
        public string Username { get; set; } = String.Empty;
        [Required(ErrorMessage = "Password is required")]
        public string password { get; set; } = string.Empty;
    }
}
```

****

The App User is:

```
using Microsoft.AspNetCore.Identity;

namespace JLokaAuthentication.Authentication
{
    public class AppUser: IdentityUser
    {
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string ProfilePicture {  get; set; } = string.Empty;
    }
}
```

****

```
namespace JLokaAuthentication.Authentication
{
    public static class AppRoles
    {
        public const string Administrator = "Administrator";
        public const string User = "User";
        public const string VipUser = "VipUser";
    }
}
```

****

```
using System.ComponentModel.DataAnnotations;

namespace JLokaAuthentication.Authentication
{
    public class AddOrUpdateAppUserModel
    {
        [Required(ErrorMessage = "Username is required")]
        public string UserName { get; set; } = String.Empty;
        [Required(ErrorMessage = "Password is required")]
        public string Password { get; set; } = String.Empty;
        [EmailAddress]
        [Required(ErrorMessage ="Email is Required")]
        public string Email { get; set; } = String.Empty;
    }
}
```

****